



# Port City International University

*Excellence in Higher Education*

## **Report on**

## **Report(Final)**

Report No : 01-07

Course code : CSE 422

Course Title : Computer Graphics Sessional

## **Submitted By**

Name of Student : Md. Hasib Uddin

Batch No : 30<sup>th</sup> (B1)

ID No : CSE 030 07861

Department of Computer Science and Engineering

## **Submitted To**

Name of Lecturer : Eshita Chowdhury

Department : CSE

Port City International University

## **Date of Submission:**

14/05/2026

# Index

| Serial No: | Report Name                                                                                                   | Page |
|------------|---------------------------------------------------------------------------------------------------------------|------|
| 01         | Implement a circle with center in rectangle (Draw rectangle -> where circle inside -> color as National Flag) |      |
| 02         | Implementation of Basic Transformations (Translation, Rotation, and Scaling) Using Graphics in C++            |      |
| 03         | Implementation of Cohen-Sutherland Line Clipping Algorithm Using Graphics in C++                              |      |
| 04         | Implementation of Bezier Curve Drawing Using Graphics in C++                                                  |      |
| 05         | Implement Cartoon Emoji using C++ language.                                                                   |      |
| 06         | Implementation of Chain of Circles Using Graphics in C++                                                      |      |
| 07         | Implementation of Concentric Circle Drawing Using Graphics in C++                                             |      |

## Report NO: 01

**Report Name:** Implement a circle with center in rectangle (Draw rectangle -> where circle inside -> color as National Flag)

**Introduction:** Computer graphics allows us to create real-world objects using basic geometric shapes and pixel-level drawing techniques. In this program, the Bangladesh flag is constructed using a combination of a filled rectangle (green background), a circle (red disc), and a flag stand. The circle is drawn using the Bresenham Circle Drawing Algorithm, which is an efficient integer-based method for rendering circles. The program demonstrates how mathematical algorithms and simple graphics functions can be combined to create structured visual designs.

**Objective:** The main objective of this implementation is to:

- To understand the Bresenham Circle Drawing Algorithm.
- To apply symmetry-based pixel plotting for circle generation.
- To construct a graphical representation of the Bangladesh flag.
- To use basic graphics functions like bar(), floodfill(), and setfillstyle().
- To combine multiple shapes into a single composite object.

## Source Code:

```
#include <graphics.h>
#include <conio.h>

void plotCirclePoints(int xc, int yc, int x,
int y, int color) {
    putpixel(xc + x, yc + y, color);
    putpixel(xc - x, yc + y, color);
    putpixel(xc + x, yc - y, color);
    putpixel(xc - x, yc - y, color);
    putpixel(xc + y, yc + x, color);
    putpixel(xc - y, yc + x, color);
    putpixel(xc + y, yc - x, color);
    putpixel(xc - y, yc - x, color);
}

void drawCircle(int xc, int yc, int r, int
color) {
    int x = 0, y = r;
    int p = 3 - 2 * r;

    while (x <= y) {
        plotCirclePoints(xc, yc, x, y, color);
        if (p < 0)
            p = p + 4 * x + 6;
        else {
            p = p + 4 * (x - y) + 10;
            y--;
        }
        x++;
    }
}

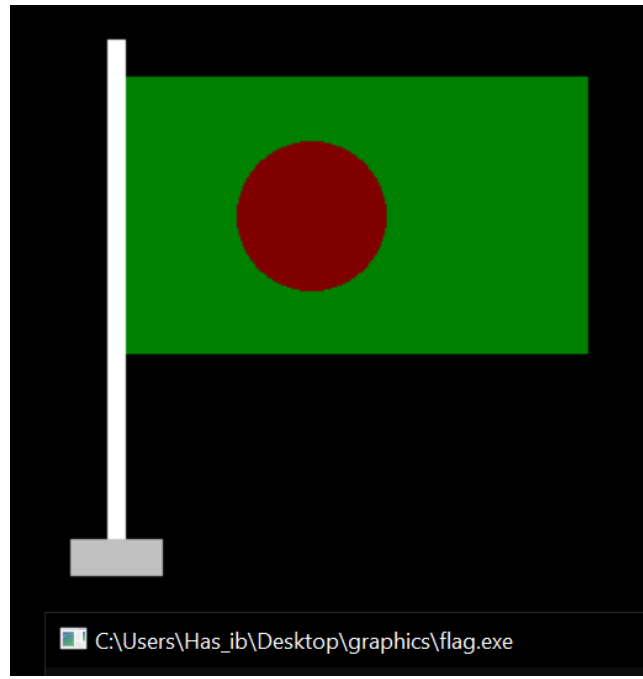
int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm,
"C:\\TurboC3\\BGI");
    setfillstyle(SOLID_FILL, GREEN);
```

```

bar(150, 100, 400, 250);
drawCircle(250, 175, 40, RED);
setfillstyle(SOLID_FILL, RED);
floodfill(250, 175, RED);
setfillstyle(SOLID_FILL, WHITE);
bar(140, 80, 150, 350);
setfillstyle(SOLID_FILL, LIGHTGRAY);
bar(120, 350, 170, 370);
getch();
closegraph();
return 0;
}

```

### Output:



**Conclusion:** This program successfully demonstrates how the Bresenham Circle Algorithm can be used in practical graphical applications. By combining pixel-based circle generation with basic shape drawing functions, a complete flag structure is created. The implementation improves understanding of symmetry-based algorithms, raster graphics concepts, and object composition in computer graphics using C++.

## Report NO: 02

**Report Name:** Implementation of Basic Transformations (Translation, Rotation, and Scaling) Using Graphics in C++

**Introduction:** Basic Transformation is an important concept in computer graphics used to manipulate graphical objects in a two-dimensional coordinate system. Common transformations include translation, rotation, and scaling. Translation moves an object from one position to another, rotation turns the object around a fixed point, and scaling changes the size of the object. This implementation uses the graphics.h library in C++ to demonstrate these three transformations on a rectangle using graphical rendering techniques.

**Objective:** The main objective of this implementation is to:

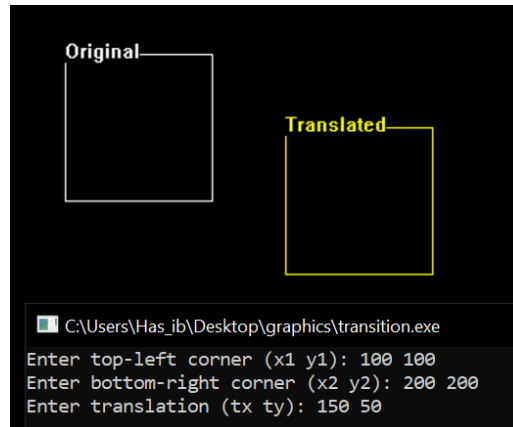
- To understand the concept of 2D geometric transformations.
- To implement translation, rotation, and scaling on a rectangle.
- To visualize object transformation using graphics functions.
- To apply mathematical formulas for coordinate transformation.
- To compare original and transformed graphical objects.

### Source Code (Transition):

```
#include <graphics.h>
#include <iostream>
using namespace std;
int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, (char*)"");
    int x1, y1, x2, y2;
    int tx, ty;
    cout << "Enter top-left corner (x1 y1): ";
    cin >> x1 >> y1;
    cout << "Enter bottom-right corner (x2 y2): ";
    cin >> x2 >> y2;
    setcolor(WHITE);
    rectangle(x1, y1, x2, y2);
    outtextxy(x1, y1 - 10, (char*)"Original");

    cout << "Enter translation (tx ty): ";
    cin >> tx >> ty;
    int new_x1 = x1 + tx;
    int new_y1 = y1 + ty;
    int new_x2 = x2 + tx;
    int new_y2 = y2 + ty;
    setcolor(YELLOW);
    rectangle(new_x1, new_y1, new_x2, new_y2);
    outtextxy(new_x1, new_y1 - 10, (char*)"Translated");
    getch();
    closegraph();
    return 0;
}
```

## Output:



## Source Code (Rotation):

```
#include <graphics.h>
#include <stdio.h>
#include <conio.h>
#include <math.h>

void drawRect(int x[], int y[]) {
    for (int i = 0; i < 4; i++) {
        line(x[i], y[i], x[(i+1)%4], y[(i+1)%4]);
    }
}

int main() {
    int gd = DETECT, gm;

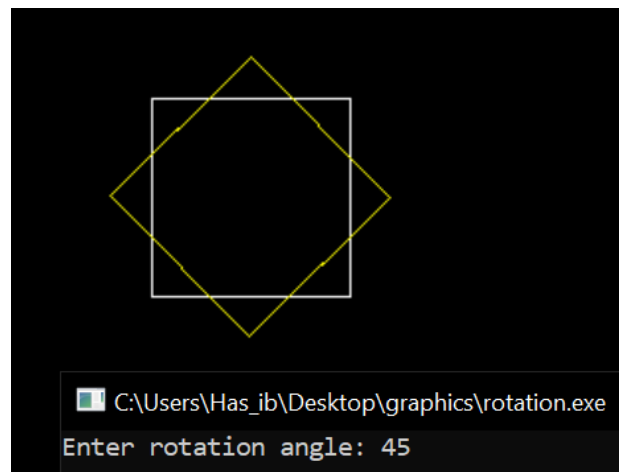
    initgraph(&gd, &gm,
        "C:\\Turboc3\\BGI");

    int x[4] = {100, 200, 200, 100};
    int y[4] = {100, 100, 200, 200};
    setcolor(WHITE);
    drawRect(x, y);
    int h = (x[0] + x[2]) / 2;

    int k = (y[0] + y[2]) / 2;

    float angle;
    printf("Enter rotation angle: ");
    scanf("%f", &angle);
    float rad = angle * 3.1416 / 180.0;
    int x_new[4], y_new[4];
    for (int i = 0; i < 4; i++) {
        x_new[i] = h + (x[i]-h)*cos(rad) - (y[i]-k)*sin(rad);
        y_new[i] = k + (x[i]-h)*sin(rad) + (y[i]-k)*cos(rad);
    }
    setcolor(YELLOW);
    drawRect(x_new, y_new);
    getch();
    closegraph();
    return 0;
}
```

### Output:



### Source Code (Scaling):

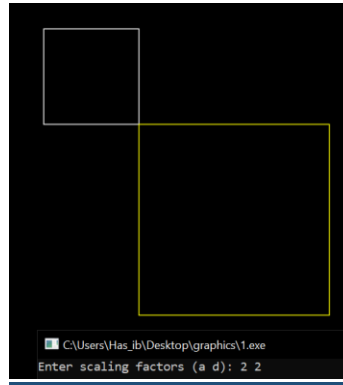
```
#include <graphics.h>
#include <stdio.h>
#include <conio.h>

void drawRect(int x[], int y[]) {
    for (int i = 0; i < 4; i++) {
        line(x[i], y[i], x[(i+1)%4], y[(i+1)%4]);
    }
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm,
        "C:\\Turboc3\\BGI");
    int x[4] = {100, 200, 200, 100};
    int y[4] = {100, 100, 200, 200};
    setcolor(WHITE);

    drawRect(x, y);
    float a, d;
    printf("Enter scaling factors (a d): ");
    scanf("%f %f", &a, &d);
    int x_new[4], y_new[4];
    for (int i = 0; i < 4; i++) {
        x_new[i] = x[i] * a;
        y_new[i] = y[i] * d;
    }
    setcolor(YELLOW);
    drawRect(x_new, y_new);
    getch();
    closegraph();
    return 0;
}
```

### Output:



**Conclusion:** This implementation successfully demonstrates the basic transformation techniques in computer graphics using C++. Translation moves the object to a new position, rotation changes its orientation around a fixed point, and scaling modifies its size. These transformations are fundamental operations in graphical applications, animation systems, game development, and computer-aided design.



## Report NO: 03

**Report Name:** Implementation of Cohen-Sutherland Line Clipping Algorithm Using Graphics in C++.

**Introduction:** The Cohen-Sutherland Line Clipping Algorithm is a widely used technique in computer graphics for clipping lines within a rectangular clipping window. The algorithm determines whether a line is completely visible, partially visible, or completely outside the clipping region by using region codes. It efficiently calculates intersection points and removes unnecessary portions of lines outside the viewing area. This implementation uses the graphics.h library in C++ to visually demonstrate line clipping using a rectangular clipping window.

**Objective:** The main objective of this implementation is to:

- To understand the working principle of the Cohen-Sutherland Line Clipping Algorithm.
- To classify lines as fully inside, partially visible, or fully outside.
- To use region codes for detecting clipping conditions.
- To calculate intersection points with clipping boundaries.
- To visualize line clipping within a rectangular window using graphics functions.

## Source Code:

```
#include <graphics.h>
#include <iostream>
#include <math.h>
using namespace std;
int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    int x1, y1, x2, y2, x3, y3, x4, y4;
    cout << "Enter 4 control points:\n";
    cout << "Point 1 (x y): ";
    cin >> x1 >> y1;
    cout << "Point 2 (x y): ";
    cin >> x2 >> y2;
    cout << "Point 3 (x y): ";
    cin >> x3 >> y3;
    cout << "Point 4 (x y): ";
    cin >> x4 >> y4;
    circle(x1, y1, 3);
    circle(x2, y2, 3);
    circle(x3, y3, 3);
    circle(x4, y4, 3);
    for(float t = 0; t <= 1; t += 0.001)
    {
        float xt =
            pow(1 - t, 3) * x1 +
            3 * t * pow(1 - t, 2) * x2 +
            3 * t * t * (1 - t) * x3 +
            t * t * t * x4;
        float yt =
            pow(1 - t, 3) * y1 +
            3 * t * pow(1 - t, 2) * y2 +
            3 * t * t * (1 - t) * y3 +
            t * t * t * y4;
```

```

putpixel(xt, yt, WHITE);
}
getch();

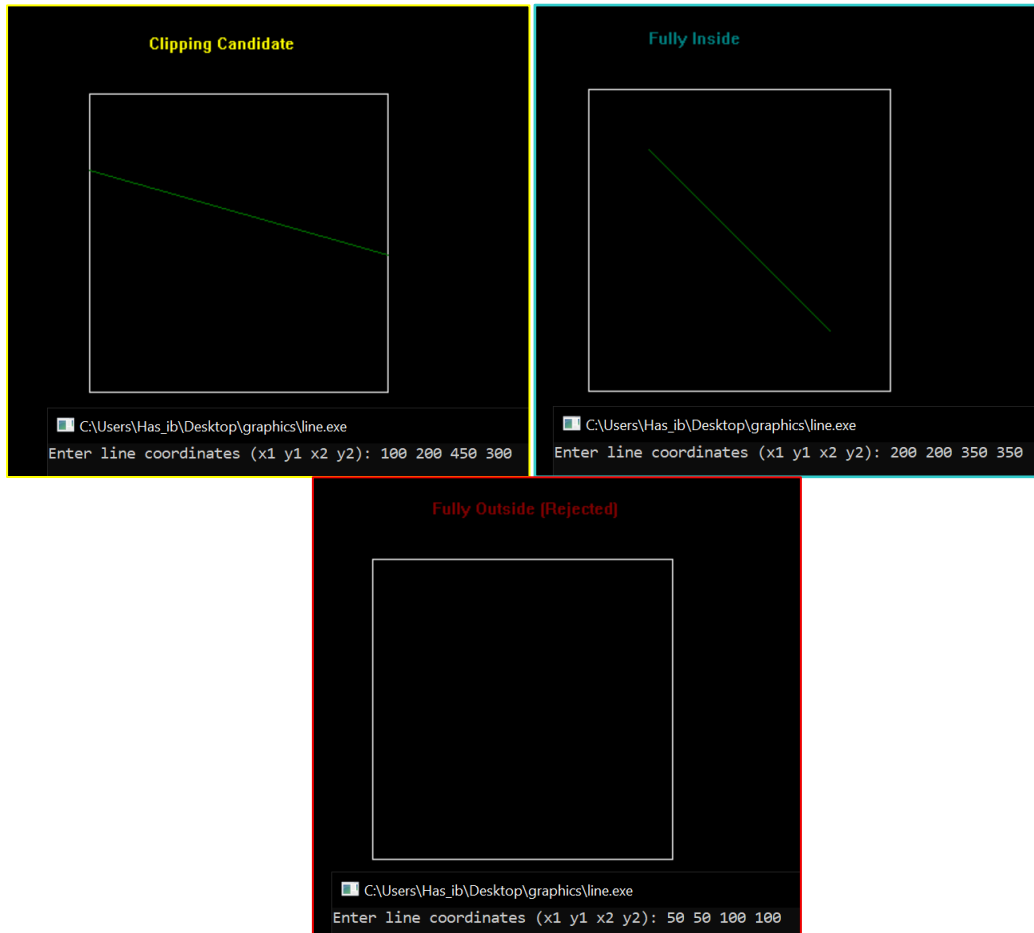
```

```

closegraph();
return 0;
}

```

### Output:



**Conclusion:** This program successfully demonstrates the implementation of the Cohen-Sutherland Line Clipping Algorithm using graphics in C++. By using region codes and intersection calculations, the algorithm efficiently determines visible portions of a line within a clipping window. The implementation helps in understanding clipping techniques, graphical rendering optimization, and fundamental concepts of computer graphics systems.

## Report NO: 04

**Report Name:** Implementation of Bezier Curve Drawing Using Graphics in C++.

**Introduction:** Bezier Curve is a mathematical curve widely used in computer graphics, animation, and design applications for generating smooth curves. The curve is controlled by a set of control points that determine its shape and direction. A cubic Bezier Curve uses four control points and calculates intermediate points using polynomial equations. This implementation uses the graphics.h library in C++ to generate and display a smooth Bezier curve based on user-defined control points.

**Objective:** The main objective of this implementation is to:

- To understand how Bezier curves are generated using mathematical interpolation.
- To implement a cubic Bezier Curve using four control points.
- To calculate curve points using parameter t.
- To visualize smooth curve generation using putpixel().
- To understand the role of control points in shaping the curve.

### Source Code:

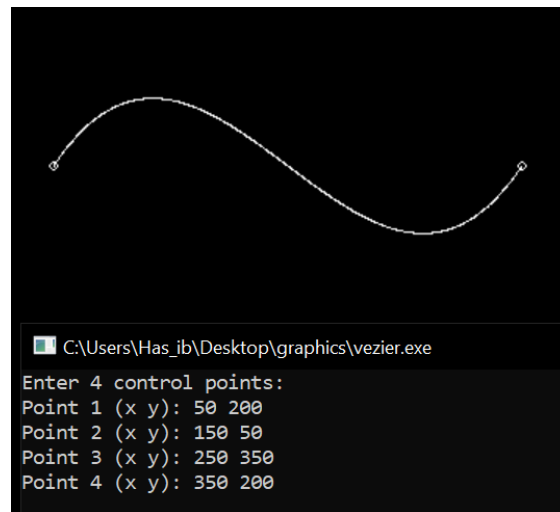
```
#include <graphics.h>
#include <iostream>
#include <math.h>
using namespace std;
int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    int x1, y1, x2, y2, x3, y3, x4, y4;
    cout << "Enter 4 control points:\n";
    cout << "Point 1 (x y): ";
    cin >> x1 >> y1;
    cout << "Point 2 (x y): ";
    cin >> x2 >> y2;
    cout << "Point 3 (x y): ";
    cin >> x3 >> y3;
    cout << "Point 4 (x y): ";
    cin >> x4 >> y4;
    circle(x1, y1, 3);
    circle(x2, y2, 3);
    circle(x3, y3, 3);
    circle(x4, y4, 3);
    for(float t = 0; t <= 1; t += 0.001)
    {
        float xt =
            pow(1 - t, 3) * x1 +
            3 * t * pow(1 - t, 2) * x2 +
            3 * t * t * (1 - t) * x3 +
            t * t * t * x4;
        float yt =
            pow(1 - t, 3) * y1 +
            3 * t * pow(1 - t, 2) * y2 +
            3 * t * t * (1 - t) * y3 +
            t * t * t * y4;
        putpixel(xt, yt, WHITE);
    }
    getch();
}
```

```
closegraph();
```

```
return 0;
```

```
}
```

### Output:



**Conclusion:** This program successfully demonstrates the implementation of a cubic Bezier Curve using graphics in C++. By using mathematical interpolation formulas and control points, the program generates a smooth curve efficiently. The implementation helps in understanding curve modeling, graphical rendering, and the importance of Bezier curves in computer graphics and animation systems.

## Report NO: 05

**Report Name:** Implement Cartoon Emoji using C++ language.

**Introduction:** Computer graphics allows the creation of visual objects using basic geometric shapes such as circles, arcs, and lines. A smiley emoji is a simple graphical representation of a face using these primitive shapes. In this program, the graphics.h library is used to draw a face, eyes, and a smile arc. The implementation demonstrates how basic drawing functions can be combined to create meaningful graphical objects.

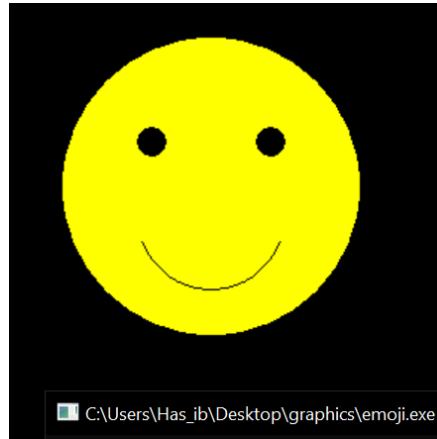
**Objective:** The main objective of this implementation is to:

- To understand the use of basic graphics functions like circle(), arc(), and floodfill().
- To create a simple emoji using geometric shapes.
- To learn how color filling works in graphics programming.
- To combine multiple shapes into a single graphical object.
- To visualize object rendering in a graphics window.

### Source Code:

```
#include <graphics.h>
#include <iostream>
using namespace std;
int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, (char*)"");
    setcolor(YELLOW);
    setfillstyle(SOLID_FILL, YELLOW);
    circle(300, 200, 100);
    floodfill(300, 200, YELLOW);
    setcolor(BLACK);
    setfillstyle(SOLID_FILL, BLACK);
    circle(260, 170, 10);
    floodfill(260, 170, BLACK);
    circle(340, 170, 10);
    floodfill(340, 170, BLACK);
    setcolor(BLACK);
    arc(300, 220, 200, 340, 50);
    outtextxy(250, 350, (char*)"Happy
    Emoji");
    getch();
    closegraph();
    return 0;
}
```

**Output:**



**Conclusion:** This program successfully demonstrates how basic graphics functions can be used to construct a simple emoji. By combining circles, arcs, and color filling techniques, a meaningful graphical representation is created. This implementation helps in understanding shape composition, color handling, and basic drawing operations in computer graphics using C++.

## Report NO: 06

### Report Name: Implementation of Chain of Circles Using Graphics in C++.

**Introduction:** A chain of circles is a graphical arrangement where multiple circles are drawn sequentially side by side while maintaining equal spacing between them. In computer graphics, this type of implementation helps in understanding shape positioning, looping structures, and color manipulation. This program uses the graphics.h library in C++ to draw a series of connected circles with user-defined colors. Each circle is positioned dynamically using coordinate calculations to create a chain-like structure.

**Objective:** The main objective of this implementation is to:

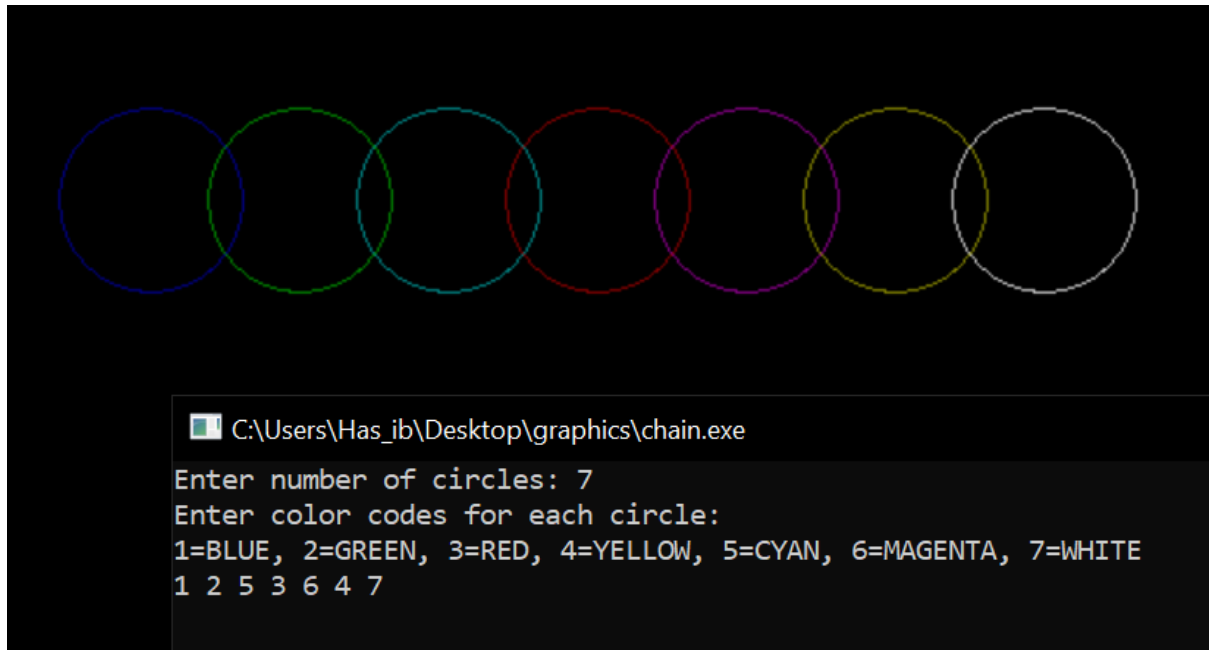
- To understand how multiple circles can be drawn using loops.
- To implement circle generation using the circle() function in graphics programming.
- To take user input for the number of circles and their colors.
- To apply different colors using the setcolor() function.
- To calculate circle positions dynamically for creating a chain effect.

### Source Code:

```
#include <graphics.h>
#include <iostream>
using namespace std;
int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    int n;
    cout << "Enter number of circles: ";
    cin >> n;
    int colors[n];
    cout << "Enter color codes for each circle:\n";
    cout << "1=BLUE, 2=GREEN, 3=RED, 4=YELLOW, 5=CYAN, 6=MAGENTA, 7=WHITE\n";

    for (int i = 0; i < n; i++)
    {
        cin >> colors[i];
    }
    int x = 100, y = 200;
    int r = 40;
    for (int i = 0; i < n; i++)
    {
        setcolor(colors[i]);
        circle(x + i * (r * 1.6), y, r);
    }
    getch();
    closegraph();
    return 0;
}
```

### Output:



**Conclusion:** This program successfully demonstrates the implementation of a chain of circles using graphics in C++. By combining loops, coordinate calculations, and color functions, the program efficiently generates multiple circles in a sequential arrangement. The implementation improves understanding of graphical object placement, dynamic input handling, and basic computer graphics concepts.



## Report NO: 07

**Report Name:** Implementation of Concentric Circle Drawing Using Graphics in C++.

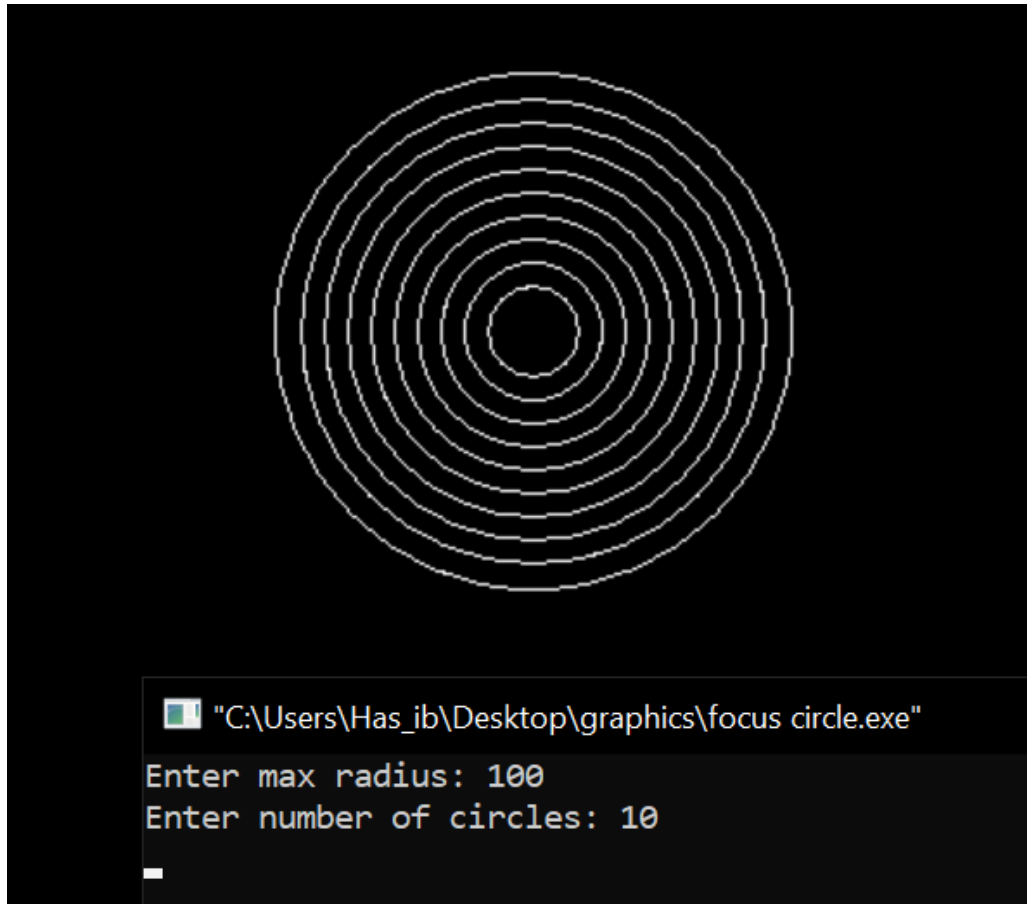
**Introduction:** Concentric circles are circles that share the same center point but have different radius. In computer graphics, concentric circles are commonly used to understand geometric drawing, coordinate positioning, and graphical rendering techniques. This program uses the graphics.h library in C++ to draw multiple circles inside a maximum radius based on user input. The implementation dynamically calculates the radius of each circle and displays them from larger to smaller size while maintaining the same center point.

**Objective:** The main objective of this implementation is to:

- To understand how concentric circles are generated using computer graphics.
- To implement circle drawing using the circle() function in graphics.h.
- To take dynamic user input for radius and number of circles.
- To calculate equal spacing between circles using arithmetic operations.
- To visualize multiple circles sharing the same center point.

### Source Code:

```
#include <graphics.h>
#include <iostream>
using namespace std;
int main()
{
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    int radius, n;
    cout << "Enter max radius: ";
    cin >> radius;
    cout << "Enter number of circles: ";
    cin >> n;
    int cx = getmaxx() / 2;
    int cy = getmaxy() / 2;
    float step = (float)radius / (n + 1);
    for(int i = 0; i < n; i++)
    {
        int r = radius - i * step;
        if(r > 0)
            circle(cx, cy, r);
    }
    getch();
    closegraph();
    return 0;
}
```

**Output:**

**Conclusion:** This program successfully demonstrates the implementation of concentric circle drawing using graphics in C++. By using loops, arithmetic calculations, and the `circle()` function, multiple circles are generated efficiently with the same center point. The implementation helps in understanding graphical coordinate systems, geometric drawing concepts, and dynamic user input handling in computer graphics applications.